

NPC Behavior in Games Using Unity ML-Agents: A Reinforcement Learning Approach

Revat Meshram

Dept. of Computer Science Engineering
(Artificial Intelligence and Machine Learning), Vishwakarma Institute of Information Technology, Pune, India
revat.meshram@gmail.com

Sachith Krishna

Dept. of Computer Science Engineering
(Artificial Intelligence and Machine Learning), Vishwakarma Institute of Information Technology, Pune, India
sachit.krishna@gmail.com

Omkaresh Kulkarni

Dept. of Computer Science Engineering
(Artificial Intelligence and Machine Learning), Vishwakarma Institute of Information Technology, Pune, India
omkaresh.kulkarni@viit.ac.in

Rutuja Rajendra Patil

Dept. of Computer Science Engineering
(Artificial Intelligence and Machine Learning), Vishwakarma Institute of Information Technology, Pune, India
rutujapat@gmail.com

Gagandeep Kaur

Dept. of Computer Science Engineering
(Artificial Intelligence and Machine Learning), Vishwakarma Institute of Information Technology, Pune, India
gagandeep.kaur@sitnagpur.siu.edu.in

Shruti Maheshwari

Symbiosis Institute of Technology, Pune,
Symbiosis International (Deemed University), Pune, India
shruti.maheshwari@sitpune.edu.in

Abstract— Machine learning is having a significant impact on video games, from speeding up the development of new games to the development of new methods in gaming. This study discusses the impact of using ML models in the development and improvement of video games. Examples cited in this paper relate to the ways in which NPCs behave. With the aid of Unity's ML-Agents toolkit, developers can create NPCs that respond to different strategies in real time. This makes the games more exciting and complex. The paper also outlines the possibilities of single-player games, such as first-person shooters and a role-playing game, to notice NPC behavior during the gameplay. In these gaming styles, RL frameworks can be applied to NPCs to enable them to learn and adapt to the environment to create an interactive experience for the player. The conclusions put an emphasis on the evolution of video games with the advent of ML tools; as predicted, NPC behavior becomes more intuitive and less expected.

Keywords— Unity, ML-Agents, NPC, Video Game, Reinforcement Learning.

I. INTRODUCTION

Non-Playable Characters [NPC's] in Video Games have evolved drastically in the past couple of years. The credit for this evolution goes completely to the advancement in Artificial Intelligence and Machine Learning. Unity ML agents, a powerful and innovative toolkit developed by Unity Technologies has allowed us to further dive into the sea of possibilities of machine learning and reinforcement learning in game development. Unity ML agents is gaining traction and revolutionizing the way games are developed and how AI is researched. NPCs play a crucial role in delivering the player the immersive gaming experience that they want to experience in the virtual world of gaming. The role of Artificial Intelligence and machine Learning has significantly improved the modern game development community, more innovative capabilities and features has allowed the gaming industry to develop more and more advanced NPC algorithms. Most of the NPCs in games are mostly pre-defined and more predictable hence leads to repetitive gameplay this significantly brings down the players engagement with the game. This is a significant problem especially when the game is a single player game where the NPC interaction contributes to a major part of the overall gaming experience.

With the rise of Artificial Intelligence and Machine Learning in the picture the whole industry has taken a massive

turn on how we can develop more intuitive and adaptive NPCs in the game. Especially in Reinforcement learning where the NPC is trained by the data from the player gameplay in real-time. This provides a continuously evolving challenge for the player and hence makes the game more interesting and more engaging. Unity 3-D is one such open sourced game development toolkit which has built in ML tools such as ML-agents which is specifically used to train agents in a single or multi-agent scenario and implement ML algorithms in pre-made environments.

A. Units

Machine Learning: It is used to focus on developing programs that are capable of performing tasks on their own by learning, as contrasting to the conventional methods of developing games.

Neural Networks: Neural networks act like real neurons present in human body. There are three main layers in these neural networks which are input layer, hidden layer, and output layer. The data is given to the input layer and the computation takes place in the hidden layers and the results are generated at the output layer.

Reinforcement Learning: Reinforcement Learning is an ML field, which is used in the proposed research for teaching an agent to take decisions in the game environment to earn Rewards. RL uses sensors to check the state of the agent and assigns it a set of actions to perform using actuators.

B. Background & Problem Motivation

In almost every video game, Non-Playable Character (NPCs) are important for creating engaging experiences. However, traditional scripting methods for NPC behavior have limited choices and are inefficient to implement, especially on large scale when the game is large the number of permutations and combinations are also large therefore it is harder to implement them via pre-defined coding. This research goes into the depths of Machine Learning used in creating NPCs.

Our research is mainly focused on exploring how Unity ML-Agents can help us develop more adaptive NPC behaviors in video games. Using Reinforcement learning to create NPCs behaviors that adapt in real time by learning and identifying patterns from player interaction and adapt their strategies leading to a more immersive gaming experience. We will be

focusing on single player games such as First Person Shooting Games[FPSs] and Role Playing Games[RPGs] where the NPC behavior has a major significant impact..

The main objective of this research paper is to practically demonstrate how Reinforced Learning and Machine Learning can be used to develop NPCs that not only just reacts to player actions but dynamically anticipate and predict player strategies to provide the player a more exiting gameplay experience. By do the following study we aim to provide a very strong foundation for the future of game development that seeks to integrate more intelligent and unpredictable NPCs.

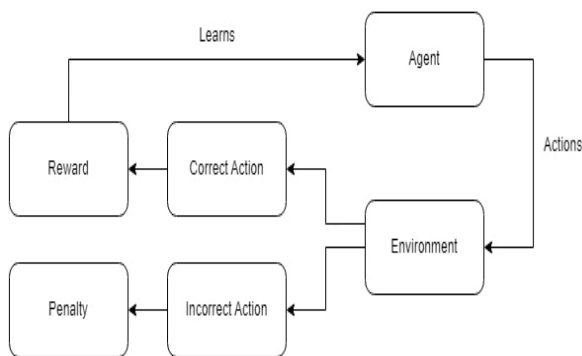


Fig. 1. Agent Interaction with environment in RL

II. LITERATURE REVIEW

Phan et al. [1] gives insight into the integration of AI in video game development and highlights how it has transformed the behavior of NPCs in video games. The authors have found out that the traditional rule-based models are getting replaced by more dynamic AI-driven NPCs. These continuously changing NPCs use real-time updates to adapt to the players' gameplays to create a better gaming experience and player engagement.

Ghanchi [2] reviews several possibilities that may be developed in Reinforcement Learning for the video game development area, mainly within the training of NPCs. He reported how RL can be used to make the NPCs learn by watching the player move with increased capability in terms of adaptability and becoming unpredictable to the players.

Such a relative study was conducted by Savit et al. [3] on simulated autonomous driving using Unity's ML-Agents framework, which coincidentally is the same used for training NPCs for games. The research done at their hands shows that reinforcement learning can be applied to train NPCs guiding complex environments themselves, similar to how self-driving cars learn to navigate life-like situations.

Duncan [4] researched the usage of RL, especially for training in-game NPCs. He concludes that with RL, NPCs can learn much more sophisticated and intuitive behaviors-like reasoning strategically and adapting in real-time to player tactics-that makes a huge difference in depth to gameplay.

Bruegelmans [5] researched the use of machine learning in game development with Unity. He investigates how Unity's ML-Agents can be used in developing even more intelligent NPCs, in that their responses would be much more tailored to player behaviors-so fewer predictions could be made from them.

Hakansson and Froberg [6] integrated machine learning to program an advanced dynamic NPC behavior in Unity 3D. The use of RL enabled the NPCs to learn identification of patterns, execution of tasks, and decision-making previously hardcoded through other means.

Namerforslund [7] investigated adversarial machine learning applications in video games. The study further showed that adversarial approached reinforcement learned NPCs can learn not just from the players but also to learn to anticipate and counter those very same player tactics, adding a new level of difficulty to the games.

Baby and Goswami proposed an AI agent for the Connect-4 game using Unity3D and concepts of machine learning. Their work showed that the RL-trained agent could learn how to adapt to any game state and hence be a more competitive and unpredictable opponent for human players [8].

Almeida et al. [9] relied on the use of RL as a method for training agents within the multiplayer FPS game genre. According to their findings, the RL-trained agents make the multiplayer experience much more dynamic since they continuously adapt to the human players' strategies and thereby become a force to be reckoned with.

Chincholkar et al. [10], in their paper, focused on improving the behavior of NPCs through machine learning algorithms in single-player games. They chose Unity ML-Agents for training NPCs that showed their ability to adapt according to the player's moves; hence, it makes the repeat factor of the games more interesting with different experiences during subsequent plays.

Real-Time Strategy Adjustments: In many games, NPCs are programmed in such a way that they can perform with static activities or strategies, which players can easily exploit or cheat once they learn the pattern. Also, some advanced NPCs use ML, they still lack the ability to adjust their behavior.

Solution: With the help of reinforcement learning, this gap can be fixed. Dynamic algorithm will help the NPC to learn through the interactions made by the player with the environment. This will help create a more user-focused experience for the user/player and will keep them engaged.

III. PROPOSED METHODOLOGY

Methodology Section of our project provides us an outline about the approach that we have used to investigate NPC behaviour in modern games using the Unity ML -Agents & Reinforcement learning. This methodology aims to provide a streamlined process that involves setting up appropriate environment scene and implementation of RL algorithms.

A. Unity ML – Agents Overview

The Unity ML-Agents Toolkit is an open source package for Unity 3D that allows games or simulations to be used as training environment for implementing machine learning algorithms. It allows the user to create more complex and interactive scenes where the agent can learn from past experience.

- **Academy:** Academy manages the training environment, includes setting the environment and handling interactions between agents and the environment

- **Agent:** Represents the character in the environment that learn to perform tasks (or the entity on which we apply machine learning algorithms). Agents receive observations from the environment, take actions (specified actions/triggers) and receive rewards.
- **Brain (Behaviour):** Like the human brain, this brain is the decision-making component that uses the trained model to determine the actions an agent should take based on its observations.

The ML-Agents package consists of the primary C#SDK, used in the Unity ML-Agents Toolkit. Different reinforcement algorithms along with this package are used to create a learning environment and train the character behaviours. These trained character behaviours are allowed to use into the Unity scenes to control characters. Model Building

B. Reinforcement Learning Framework

Reinforcement Learning (RL) is a type of machine learning which uses reward based feedback mechanism to train the agents to take decisions based on the interactions with their environment. In this topic of AI NPC we use RL to train these NPCs (agents) to navigate and collect pellets within a Unity Simulation, or in easier terms Hide and Seek, where the seeker tries to find the hider [11] [12].

Some of the core components of RL includes:

- **Agent:** The character that learns and makes decisions.
- **Environment:** The scene or game world where the agent operates.
- **State:** The current situation or configuration that the agent observes.
- **Action:** The choices the agent can make that impact the environment.
- **Reward:** Feedback received from the environment to reinforce or not reinforce on wrong actions.

Proximal Policy Optimization (PPO) is a popular RL algorithm. PPO is used for handling complex, continuous action spaces and provides stable training through its approach to updating policies [13]. The algorithm involves:

Policy Network: A neural network that maps states to actions, learning to improve decision-making over time.

Value Network: Estimates the expected reward for a given state.

C. Environment Setup

The training environment is crucial for effective RL. In this study, we designed a Unity-based environment where agents must navigate and collect pellets. The setup includes:

Scene Creation: We developed a simple 3D environment that is basic for demonstration purposes. This scene setup involves a plane covered by 4 sides each by wall. This simple scene will allow us to clearly see and train our Agent (seeker) to find the scattered Pellets (hider).

Agent Configuration: Using the ML -Agents toolkit we have access to sensors and actuators. A sensor senses the conditions and provides a signal when changes occur. An

actuator receives a signal and performs physical action. The Agent uses the ML-Agent script to interact and move with changes in the environment.

D. Training Process

For training process, we start by setting up the Unity scene and installing all the necessary dependencies and packages such as Pytorch, Nvidia CUDA etc. For training we have setup a virtual python environment using CMD.

- To speed up the training process, the environment scene, which consists of a four walled area with a pellet and grabber, we duplicated approximately 60-80 times. This duplication of scene allows multiple copies of the environment to run in parallel, imagine training a dog to bark but training multiple instances of the same dog would make it faster to learn thus significantly speeding up the learning process.
- After every training iteration, the agent's performance is reviewed on all the duplicated scenes and metrics like average reward, success rate, and time taken to collect one pellet is analysed to ensure that the agent is showing a positive learning rate.
- Finally, once the agent has achieved the minimum performance requirement, it can now be used into our simulated scene. We can use the final model once the Agent (seeker) is able to find pellets (hider) almost every time. For the implementation part, the following steps are followed:
- **Environment Initialization:** As discussed in the methodology, we are using a basic 3D environment consisting of a plane which is covered on all 4 sides with a wall. This clean layout allows our agent to get trained easily and is intuitive to work upon.
- **Agent Configuration:** For our pellet grabber or hide and seek project we are using 2 basic shape, cube and sphere. Here the sphere acts as a pellet (hider) and the sphere (grabber) tries to find the pellet.
- **Agent Script:** It is the script that controls the agent and is responsible for all the actions that the agent make.
- The script can have custom parameters and reward system in order to increase the positive learning rate of our agent. This script consists of sensors and actuators and decides what action will be taken when there is change in the environment.
- **Training Process:** Once everything is setup, The ML-Agent -learn can be used to train the model (agent). As discussed in the methodology, by using multiple copies of the environment we can speed up the training process. For every correct action of the agent the agent is granted with a reward and for every incorrect action it is penalized.
- **Testing:** It is simply checking if the Agent is able to find the pellets (seeker) and the time required to find the target. From Fig 4, We can observe how the replicated system works and after the training process 1 out of 21 remains incorrect ensuring that the model is trained.

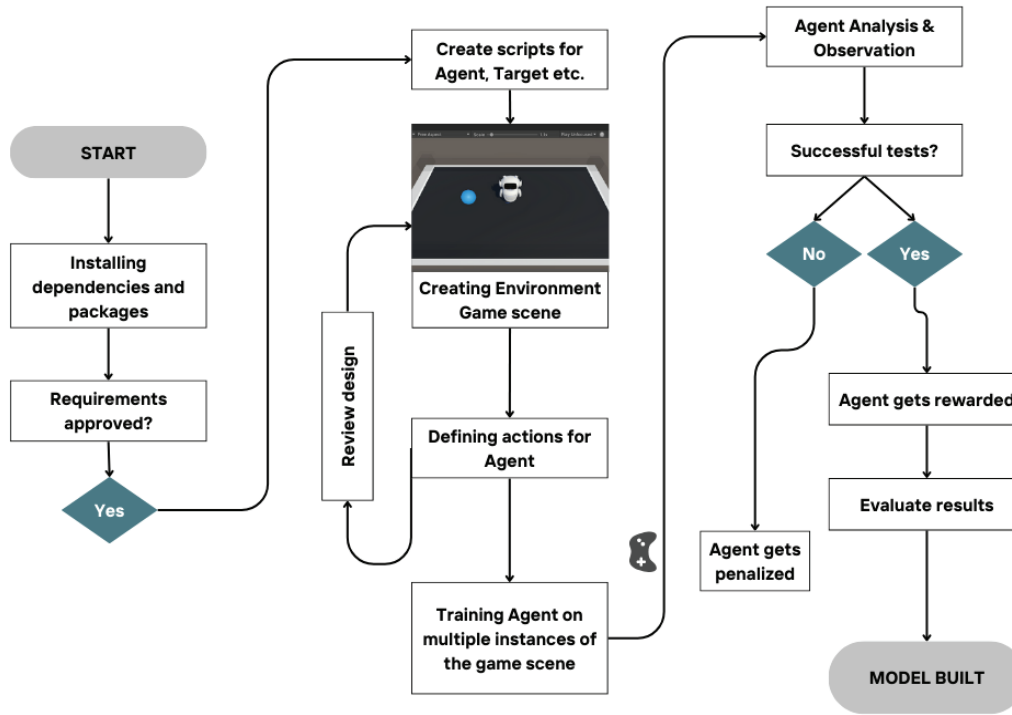


Fig. 2. Overall Flow

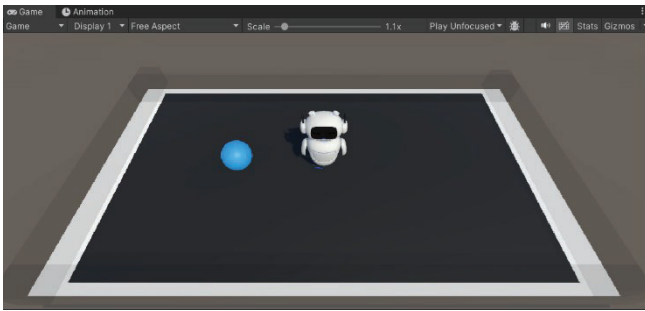


Fig. 3. Game Environment Scene

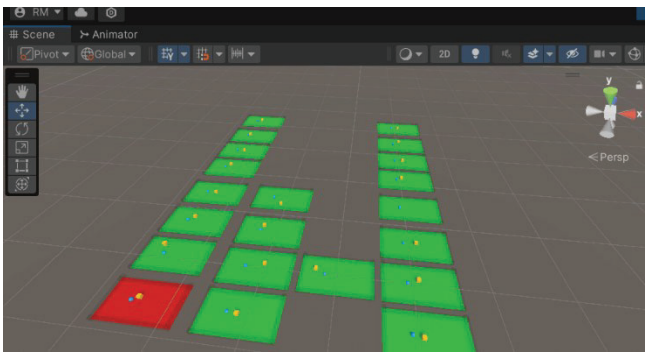


Fig. 4. Live Accuracy Demonstration – Green indicates reward & Red indicates penalty.

IV. RESULT ANALYSIS

A. Performance Metrics

In this section, different performance metrics of this project will be discussed.

Cumulative Reward: It is one of the most important Metric used in ML – Agents used for comparison [14]. Cumulative Reward simply means the total rewards gathered by the agent during a training session. For every training session, if the model performs correctly the agent receives a reward and penalty for behaving incorrectly.

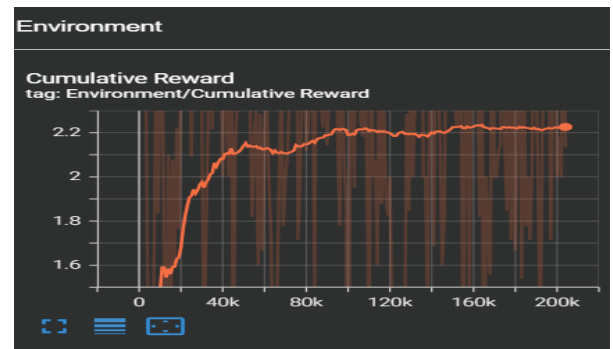


Fig. 5. Cumulative Reward Graph

From the graph in Fig 5 we can observe during the early phase of training the agent is collecting less rewards on average, we can observe this increasing over time this indicating the agent is gathering more rewards on average.

Episode Length: It is the time required for the agent to find the target, the lesser the time the faster is the agent and the more accurate is the model.

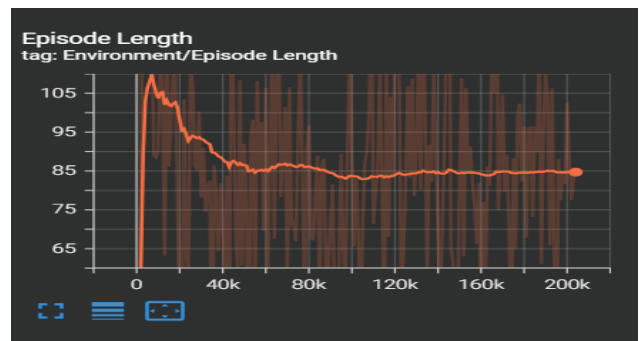
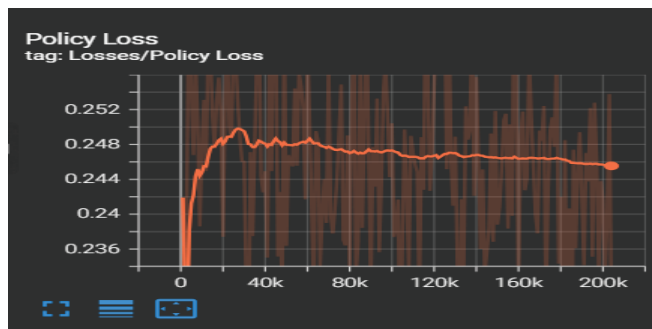


Fig. 6. Episode Length Graph

From the graph in Fig 6, we can make sure, after the model learns, the time required to find the target reduces, indicating improved accuracy.

Policy Loss: For gathering the reward, the agent uses a strategy, over time the agent learns what is the most ideal strategy to get most rewards [15-17].



Policy Loss Graph

Therefore, we can observe in the graph in Fig 7 during the early stage, the agent is trying to find the strategy and after some time, it gets stabilized as it learns more efficient strategies.

B. Training Results

Here we can see the training results from the python virtual environment itself such as reward and penalties the agent received photo.

C. Comparison with Traditional Methods

In traditional game development, NPC behaviour is typically controlled by a scripted AI, this scripted AI has finite states or behaviour trees. Also, the scripted agent would not be able to learn to new changes made in environment. These methods have been used widely in many games due to its simplicity, however this can make the game monotonous and boring. Comparison based on some parameters are as follows:

Predict vs Adapt: Traditional Method NPCs use a scripted AI or follow predetermined paths and behaviours which can make them highly predictable and this boring after some extent. Reinforcement Learning in contrast can learn to adapt in to different situations and environments therefore is more dynamic when compared to traditional methods

Complex Decision Making: Traditional Method lacks complex decision making, they require manual programming to handle various scenarios or transitions, which becomes more difficult to handle. RL allows the NPCs to make complex decisions based on its learned experience rather than predetermined rules.

Scalability: Traditional Methods, adding new behaviours require large amount of coding and understanding, and rewriting portions of code, which can be time consuming and also introduces to newer more complex bugs. RL models can be scaled easily, once the model is trained, it is ready to be applied to any agent and can adjust according to the changes made in the environment.

V. CONCLUSION

Machine learning in games helps us integrate more dynamic features and possibilities. With this research, we have discussed the application of reinforcement learning to create smart NPCs. Using Unity's ML-Agent toolkit, which allows us to use ML in Unity, the NPCs were trained to create a RL model.

With the help of this pellet and grabber game, this research has shown the potential and extent of RL to create dynamic

behaviour for NPCs. Reward-based learning: NPCs throughout their game length have improved their strategies and behaviours, resulting in more challenging gameplay.

This research has explored the efficiency of RL to create large-scale model games and has also provided a pathway for the future by taking RL to the next level and creating more dynamic features, such as procedural content generation, overall creating an even more realistic experience. Many industries have been doing their jobs to provide better experience to gamers by implementing AI and Machine Learning. Ubisoft is one such industry, who are doing their best for the mainstream use of ML in the game development.

REFERENCES

- [1] J. Ghanchi, "AI in Game Development," IT Chronicles, no. 05, pp. 7716-7719, 2021. [Online]. Available: <https://itchronicles.com/artificial-intelligence/ai-in-game-development/>
- [2] Y. Savid, R. Mahmoudi, R. Maskeliūnas, and R. Damaševičius, "Simulated Autonomous Driving Using Reinforcement Learning: A Comparative Study on Unity's ML-Agents Framework," Information, vol. 14, no. 5, 2023, doi: 10.3390/info14050290.
- [3] R. Duncan, "Using Reinforcement Learning to Train In-game Non-Player Characters (NPCs)."
- [4] A. Breugelmans, "Implementing machine learning (AI) in game development with Unity," 2021. [Online]. Available: <https://www.theseus.fi/handle/10024/496604>
- [5] C. Håkansson and J. Fröberg, "Application of machine learning to construct advanced NPC behaviors in Unity 3D," 2021.
- [6] T. Nämerforslund, "Machine Learning Adversaries in Video Games," no. C, 2021.
- [7] N. Baby and B. Goswami, "Implementing artificial intelligence agent within connect 4 using Unity3D and machine learning concepts," International Journal of Recent Technology and Engineering, vol. 7, no. 6, pp. 193-200, 2019.
- [8] P. Almeida, V. Carvalho, and A. Simões, "Reinforcement Learning as an Approach to Train Multiplayer First-Person Shooter Game Agents," Technologies, vol. 12, no. 3, pp. 1-20, 2024, doi: 10.3390/technologies12030034.
- [9] S. A. Almeida Jr. and S. Guner, "Review of artificial neural networks and a new feed-forward network for anchorage analysis in cracked concrete," International Concrete Abstracts Portal, vol. 350, pp. 54-68, 2021.
- [10] H. Technology, L. Issn, and O. Chincholkar, "Machine Learning Algorithm for NPC in Game," vol. 28, no. 12, pp. 788-795.
- [11] K. Souchleris, G. K. Sidiropoulos, and G. A. Papakostas, "Reinforcement learning in game industry—review, prospects and challenges," Applied Sciences, vol. 13, no. 4, p. 2443, 2023.
- [12] A. K. Shakya, G. Pillai, and S. Chakrabarty, "Reinforcement learning algorithms: A brief survey," Expert Systems with Applications, vol. 231, p. 120495, 2023.
- [13] S. Corecco, G. Adorni, and L. M. Gambardella, "Proximal policy optimization-based reinforcement learning and hybrid approaches to explore the cross array task optimal solution," Machine Learning and Knowledge Extraction, vol. 5, no. 4, pp. 1660-1679, 2023.
- [14] J. Guo, et al., "Polycleanse: Backdoor detection and mitigation for competitive reinforcement learning," in Proceedings of the IEEE/CVF International Conference on Computer Vision, 2023.
- [15] S. E. Li, Deep reinforcement learning, in Reinforcement Learning for Sequential Decision and Optimal Control, Singapore: Springer Nature, 2023, pp. 365-402.
- [16] H. R. Goyal, K. K. Ghanshala, and S. Sharma, "Adaptive IoT Enabled Flood Management System: An Artificial Intelligence Approach," International Journal of Computing and Digital Systems, vol. 12, no. 1, pp. 607-617, 2022, doi: 10.12785/ijcds/120149.
- [17] J. C. Babu et al., "IoT-Based Intelligent System for Internal Crack Detection in Building Blocks," Journal of Nanomaterials, vol. 2022, 2022, doi: 10.1155/2022/3947760.